

PROBLEM STATEMENT PS-D04 — “Resolved, Allegedly”

CleanProof

AI-powered verification of civic complaint closures

Tracks every complaint, checks whether ‘resolved’ is actually true, and shows where cities miss deadlines or close complaints falsely.

What this document is

The complete plan of our hackathon project: the problem, our solution, every feature, the verification logic, architecture, tech stack, data plan, demo script and a suggested slide-by-slide PPT outline. Use it as the single source of truth for building the PPT.

Contents

1. The problem	11. Frontend pages & UI
2. Our solution in one minute	12. Data strategy (images & synthetic data)
3. Who uses it (users & roles)	13. Responsible design & PS rules
4. End-to-end flow	14. Demo scenario (what judges will see)
5. Features we are building	15. How we match the judging criteria
6. The verification engine (core innovation)	16. How we build it in 24 hours
7. System architecture	17. Limitations & future scope
8. Tech stack	18. Likely judge questions
9. Database design	19. Suggested PPT outline (slide by slide)
10. Backend API endpoints	20. References

1. The problem

“A garbage complaint marked ‘resolved’ in 12 hours looks like good governance — unless the garbage is still there.” (PS-D04 tagline)

The Ministry of Housing and Urban Affairs (MoHUA) runs the **Swachhata** platform, where citizens report civic problems such as garbage dumps, blocked drains, potholes and broken public toilets, with a photo and location. Under **Swachh Bharat Mission-Urban 2.0 (SBM-U 2.0)**, every category has an official deadline (SLA), and performance feeds into the national **Swachh Survekshan** city rankings.

Official SLA deadlines (SBM-U 2.0)

Complaint category	Deadline to resolve
Garbage dump	12 hours
Street not swept	24 hours
Construction debris	72 hours
Blocked drain (our assumption, not given in the PS — team to confirm)	48 hours

What goes wrong in practice

- Complaints are marked **‘resolved’** **without anyone verifying** the fix.
- Repeat complainants are sometimes **asked to stop posting** instead of having the issue fixed.
- There is **no consistent public dashboard** of real SLA adherence by city, ward or category — only self-reported campaign numbers (e.g. one civic body reported acting on 44,500 complaints).
- **Real fraud (July 2026):** the Municipal Corporation of Gurugram terminated employees for closing complaints with **AI-faked before/after photos and GPS spoofing**.

Root cause: the reporting channel works; the **accountability loop** that checks whether a ‘resolved’ complaint was genuinely resolved does not exist.

The challenge given to us

Design and prototype a system that **(1)** tracks a civic complaint from submission to claimed resolution, **(2)** independently verifies whether the resolution evidence is credible, and **(3)** surfaces SLA-adherence patterns across wards and categories for public and official accountability.

Stakeholders

Stakeholder	What they need
Urban residents (citizens)	Easy reporting, honest status, a way to reopen fake closures
Municipal sanitary inspectors & engineers	A clear task queue and a simple way to submit proof of work
Urban Local Bodies (ULBs) & MoHUA	True SLA performance, fraud detection, ward-level accountability
Swachh Survekshan assessors	Verified, not self-reported, performance data

2. Our solution in one minute

CleanProof is a web platform that tracks every civic complaint against its official deadline, **automatically checks the ‘after’ photo** when a complaint is closed using AI and photo forensics, gives each closure a **trust score (0–100)**, sends suspicious closures to a **human reviewer**, lets citizens **reopen** fake closures, and shows everything on a **public accountability dashboard** with a map.

The key idea

Today, ‘resolved’ is just a status that anyone can set. CleanProof turns ‘resolved’ into a **claim that must be proven**. Every closure needs an ‘after’ photo, and that photo is checked five ways:

- **Is the problem still visible?** (Google Gemini vision AI)
- **Was it taken at the same place?** (GPS distance between before and after photos)
- **Was it taken after the complaint?** (photo timestamp)
- **Is it a recycled photo?** (perceptual hashing against all previous closures)
- **Does it have real camera metadata?** (EXIF check — AI-generated and edited images usually have none)

What makes it different

Normal complaint apps	CleanProof
Worker clicks ‘resolved’ — done	Closure must pass 5 automatic checks
Status is self-reported	Trust score + verdict with plain-English reasons
Citizen has no recourse	One-tap reopen with a new photo; SLA clock restarts
No visibility of real performance	Public dashboard: SLA %, false closures per ward, hotspot map
Blind to photo fraud (Gurugram case)	Detects reused, wrong-location, back-dated and metadata-less photos

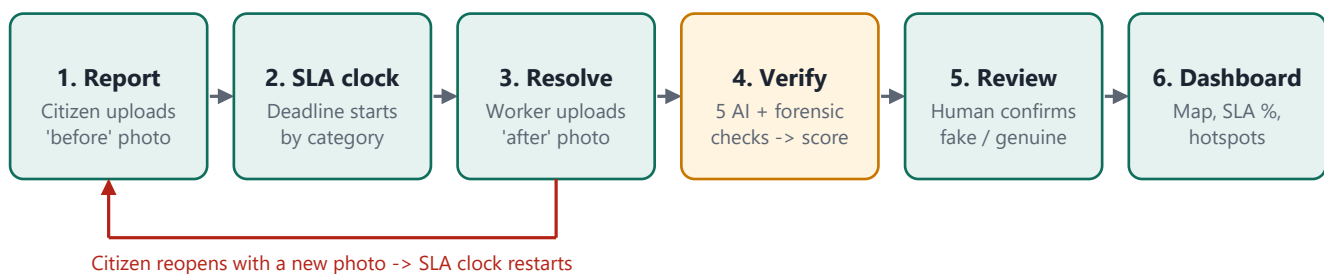
3. Who uses it (users & roles)

Role	What they do in CleanProof
Citizen	Reports an issue (photo, category, location), tracks status and SLA timer, reopens a complaint with a new photo if the problem is still there.

Role	What they do in CleanProof
Municipal worker	Sees open complaints in a task queue, fixes the issue, marks it resolved by uploading an 'after' photo. Verification runs automatically.
Human reviewer (supervisor)	Opens flagged closures, compares before/after side by side with the score and reasons, and marks each one ' Confirmed fake ' or ' Genuine '. The system never punishes anyone by itself.
Officials / public	Use the dashboard: SLA adherence by ward and category, false-closure counts, hotspot map, list of flagged complaints.

For the hackathon prototype there is **no login system** — roles are separate pages in the app. Authentication is listed under future scope.

4. End-to-end flow



Step by step

#	Step	What happens
1	Report	Citizen uploads a 'before' photo, picks a category, and the app records GPS (latitude/longitude), timestamp and ward. Complaint status: OPEN .
2	SLA clock	Deadline = complaint time + category SLA. Live status: On time → Near deadline → Breached .
3	Resolve	Worker uploads an 'after' photo and marks it resolved. Status: RESOLVED .
4	Verify	The verification engine runs 5 checks and produces a score (0–100), a verdict (VERIFIED / SUSPICIOUS / LIKELY FAKE) and a list of reasons.
5	Review	SUSPICIOUS and LIKELY FAKE closures go to the review list. A human marks them 'Confirmed fake' or 'Genuine'.
6	Reopen	If the problem is still there, the citizen uploads a new photo. Status: REOPENED , and the SLA clock restarts.
7	Dashboard	All of this feeds the dashboard: map markers, heatmap, SLA adherence %, false-closure count per ward.

5. Features we are building

We build **only these five modules**. Nothing extra — so that everything works reliably in the demo.

Module 1 — Complaint submission

- Mobile-friendly form: photo upload, category, short description
- GPS latitude/longitude captured automatically (browser location) plus ward selection
- Timestamp saved automatically; complaint gets an ID the citizen can track

Module 2 — SLA clock

- Deadline per category: garbage dump 12h, unswept street 24h, blocked drain 48h, construction debris 72h
- Live status badge: On time (green) / Near deadline (amber) / Breached (red)
- Clock restarts when a complaint is reopened

Module 3 — Resolution + automatic verification

- Worker uploads the 'after' photo to close a complaint
- 5 checks run automatically (details in Section 6)
- Result: trust score 0–100, verdict, and plain-English reasons

Module 4 — Citizen reopen + human review

- Citizen reopens a closed complaint with a new photo; reopen history is logged
- Flagged closures appear in a review list with a side-by-side before/after comparison
- Reviewer marks 'Confirmed fake' or 'Genuine' — humans make the final decision

Module 5 — Accountability dashboard

- Leaflet + OpenStreetMap map with complaint markers and a heatmap layer (hotspots)
- SLA adherence % by ward and by category (Recharts)
- False-closure count per ward, and a list of flagged complaints
- Headline numbers: total complaints, % on time, % verified, number of suspected fake closures

6. The verification engine (core innovation)

When a worker uploads an 'after' photo, the engine starts at **100 points** and deducts points for every failed check. Every deduction comes with a reason in plain English, so both citizens and officials understand why a closure was flagged.

#	Check	What it catches	How it works	Deduction
1	Issue still present?	Garbage or problem still visible after 'resolution'	Google Gemini vision AI looks at the after photo and answers yes/no with a confidence value	–50
2	Same location?	After photo taken somewhere else	Distance between before and after GPS coordinates must be under 50 metres	–25
3	Timestamp valid?	Old or back-dated photos	After-photo time (from EXIF) must be later than the complaint time	–20
4	Duplicate photo?	Same 'clean' photo reused for many complaints	Perceptual hash (imagehash) compared with all previous after photos; distance ≤ 5 = duplicate	–30
5	Camera metadata?	AI-generated, edited or downloaded images	Checks whether the photo has EXIF metadata; missing EXIF = suspicious	–10

Verdict bands

0 - 39 LIKELY FAKE	40 - 74 SUSPICIOUS	75 - 100 VERIFIED
---------------------------	---------------------------	--------------------------

0 100

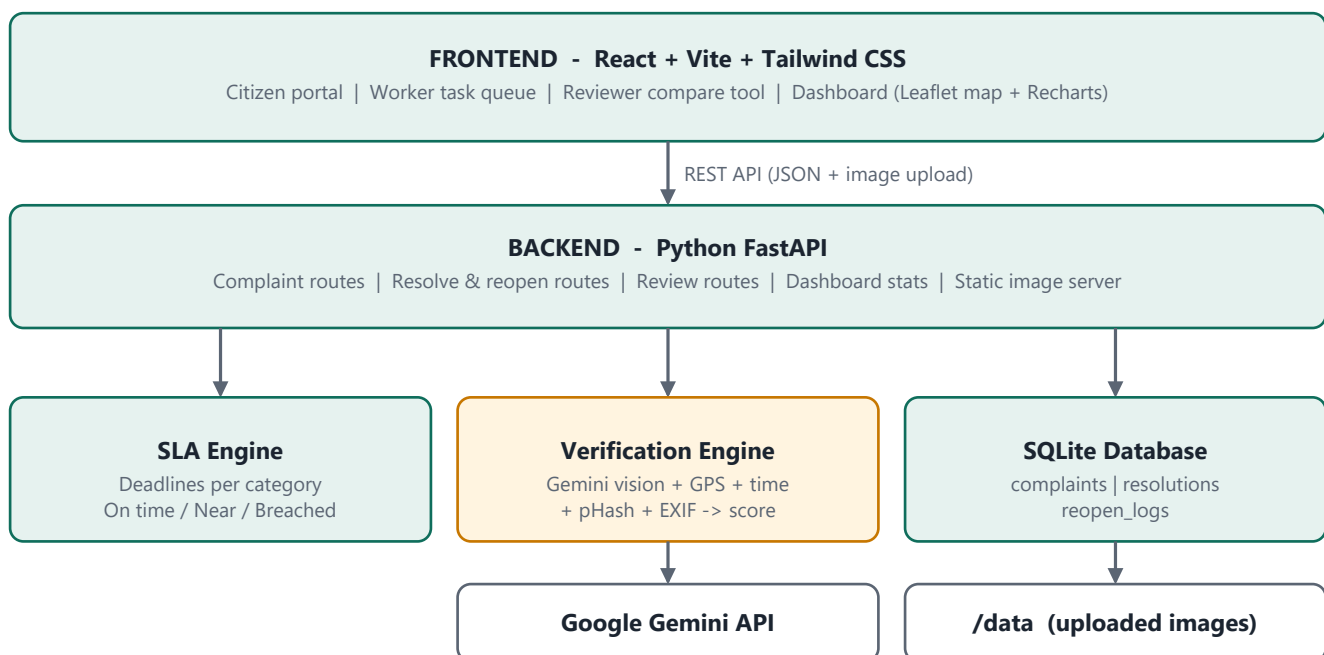
- Score never goes below 0.
- The after photo's location and time are read from its **EXIF metadata**. If the photo has no GPS or no time, that check **counts as failed** (it cannot be verified), so metadata-less fakes cannot pass.
- **VERIFIED** closures are accepted. **SUSPICIOUS** and **LIKELY FAKE** closures go to the human review list.
- **Safety net**: if the Gemini API fails (no key, quota finished, no internet), the app does **not** crash. Check 1 is marked 'unavailable', its deduction is skipped, and the other 4 checks still run.

Worked examples

Scenario	Checks failed	Score	Verdict
Genuine clean-up, same spot, real phone photo	None	100	VERIFIED
Worker re-uploads the old dirty photo	Issue present, timestamp before complaint	30	LIKELY FAKE
Clean photo of a different street	Location (>50m)	75	VERIFIED*
AI-generated clean street image	Missing EXIF, so location and time cannot be verified	45	SUSPICIOUS
Same 'clean' photo used for 3 complaints	Duplicate, location	45	SUSPICIOUS
Garbage still there, real photo at same spot	Issue present	50	SUSPICIOUS

* Edge case to tune during testing: a wrong-location photo alone scores exactly 75. If the team wants it flagged, raise the location penalty to -30 (then it scores 70 = SUSPICIOUS). Exact scores in the examples depend on which checks fail in the real run.

7. System architecture



- **Frontend** (React) talks to the **backend** (FastAPI) through a REST API.
- The backend stores data in **SQLite** and images in the **/data** folder, served as static files.

- The **SLA engine** computes deadlines and live status for every complaint.
- The **verification engine** runs the 5 checks; only check 1 calls the external **Gemini API**, the rest run locally.
- The API key is kept in a **.env** file and is never written into the code.

Project folder structure

```

resolved-allegedly/
├── PROJECT_BRIEF.md      project rules for the AI coding agent
├── backend/             FastAPI app, models, SLA + verification engines, .env
├── frontend/           React + Vite + Tailwind app
├── data/images/         before / after / fraud test images
└── scripts/            synthetic data generator, GPS/EXIF tagging

```

8. Tech stack

Layer	Technology	Why we chose it
Frontend	React + Vite	Fast to build, fast to run, huge ecosystem
Styling	Tailwind CSS	Clean, modern, responsive UI quickly
Map	Leaflet (react-leaflet) + OpenStreetMap + heatmap layer	Free, no API key, open ward maps
Charts	Recharts	Simple, good-looking dashboard charts
Backend	Python FastAPI	Simple, fast, auto-generated API docs at /docs; Python is best for images
Database	SQLite + SQLAlchemy	Zero setup, a single file, enough for a prototype
AI vision	Google Gemini API	Understands images without training a model; free tier
Image forensics	Pillow, imagehash, piexif	EXIF/GPS/timestamp reading, perceptual hashing
AI coding tool	Google Antigravity	AI agent editor used to build faster, step by step
Version control	Git	Save progress after each working step; undo mistakes
Test images	Gemini / ChatGPT image generation + editing	Create synthetic before/after pairs

9. Database design

Three tables in SQLite. (Field names are the planned design; the final code may name them slightly differently.)

Table	Main fields	Purpose
complaints	id, category, description, latitude, longitude, ward, before_image, created_at, sla_hours, sla_deadline, status (OPEN / RESOLVED / REOPENED), reopen_count	One row per complaint; drives the SLA clock
resolutions	id, complaint_id, after_image, resolved_at, exif_present, exif_time, exif_lat, exif_lng, image_hash, gemini_result, gps_distance_m, score, verdict, reasons, review_decision (Confirmed fake / Genuine), reviewed_at	One row per closure attempt, with the full verification result
reopen_logs	id, complaint_id, image, note, created_at	History of citizen reopens (used for repeat false-closure hotspots)

10. Backend API endpoints

Planned endpoints (FastAPI). The interactive documentation is available at <http://localhost:8000/docs> — useful to show judges that the backend is real.

Method	Endpoint	What it does
POST	/api/complaints	Create a complaint (photo, category, GPS, ward, description)
GET	/api/complaints	List complaints with SLA status (filters: ward, category, status)
GET	/api/complaints/{id}	One complaint with its resolutions and reopen history
POST	/api/complaints/{id}/resolve	Worker uploads after photo → verification runs → score + verdict
POST	/api/complaints/{id}/reopen	Citizen reopens with a new photo → SLA clock restarts
GET	/api/review/flagged	All SUSPICIOUS / LIKELY FAKE closures waiting for review
POST	/api/review/{resolution_id}	Reviewer decision: Confirmed fake or Genuine
GET	/api/dashboard/...	Summary numbers, SLA % by ward and category, false closures per ward, map points

11. Frontend pages & UI

Page	Used by	What it shows
Home / landing	Everyone	What CleanProof does, entry buttons for each role
Report an issue	Citizen	Mobile-friendly form: photo, category, auto GPS, ward, description
Track complaint	Citizen	Status, live SLA countdown, before/after photos, verdict, Reopen button
Task queue	Municipal worker	Open complaints sorted by deadline; upload after photo to resolve
Review (compare tool)	Reviewer	Before and after side by side, score, verdict, reasons, Confirmed fake / Genuine buttons
Dashboard	Officials / public	Headline numbers, map with markers + heatmap, SLA charts, false closures per ward, flagged list

Design principles

- Clean, modern and professional — civic green colour theme, clear typography, no clutter
- Colour-coded status everywhere: green = on time / verified, amber = near deadline / suspicious, red = breached / likely fake
- Mobile-first complaint form (citizens use phones); dashboard designed for laptop screens
- Our own name and branding — we never copy the official Swachhata look

12. Data strategy (images & synthetic data)

The PS allows only “**self-photographed or synthetic sample before/after complaint images created by the team**”. We follow this exactly.

Data needed	How we create it
'Before' images (15–20)	Generated with AI image tools (Gemini / ChatGPT), e.g. "realistic phone photo of an overflowing garbage bin on an Indian street"
'After' images	The same before image edited with AI to remove only the garbage — same place, same angle, now clean
Fraud test cases	Dirty photo re-used as the after photo; clean photo of a different street; one clean photo reused for 3 complaints; a fully AI-generated clean street
GPS and timestamps	Added to images with a Python script (piexif); some fraud images deliberately left without EXIF
Live photos	2–3 real phone photos taken at the venue (location ON) to show real EXIF/GPS checks live
Dashboard data	~200 synthetic complaints across wards and categories generated by a script
Ward map	OpenStreetMap (open data)
SLA rules	Published SBM-U 2.0 SLA guidelines

Public datasets (TACO, Kaggle) are **not listed** in the PS as permitted inputs. We will use them only if the organisers confirm it is allowed. All demo data is clearly labelled **synthetic**.

13. Responsible design & PS rules

PS rule	How CleanProof follows it
No access to or scraping of live municipal systems	Our own independent pipeline; demo uses only team-created data
Minimal personal data	No names, phone numbers or accounts are stored — only the complaint, photo and location
No automated municipal action	The system only flags. A human reviewer makes every final decision; no auto-penalties
No impersonation of official channels	Own name (CleanProof) and branding; clearly a prototype
Honest about accuracy	We report measured accuracy on our test set and show confidence and reasons, not just a verdict

14. Demo scenario (what judges will see)

#	Demo step	What the judges see
1	Citizen reports	A photo of an overflowing garbage bin is submitted; the 12-hour SLA clock starts
2	Worker closes it	The complaint is marked resolved with an 'after' photo
3	Genuine closure	A real clean-up photo scores 100 → VERIFIED (green)
4	Fake closure	The old dirty photo is re-uploaded → 'Garbage still present' → LIKELY FAKE (red)
5	AI fraud (Gurugram-style)	An AI-generated clean image → 'No camera metadata, location and time cannot be verified' → SUSPICIOUS
6	Citizen reopens	Citizen uploads a new photo; the complaint reopens and the SLA restarts
7	Human review	Reviewer compares side by side and marks 'Confirmed fake'

#	Demo step	What the judges see
8	Dashboard	This ward lights up as a red hotspot: '3 false closures this month', SLA % drops

Backup plan: record a screen video of the full demo beforehand, in case the internet or the Gemini API fails during judging.

15. How we match the judging criteria

PS evaluation focus	How CleanProof addresses it
Problem understanding	Grounded in SBM-U 2.0 SLAs, Swachhata app issues and the real Gurugram AI-photo fraud case
Technical depth (especially verification logic)	5-layer verification: vision AI + GPS + timestamp + perceptual hashing + EXIF forensics, with a transparent scoring model
Visual / UX clarity of the dashboard	Map with heatmap, colour-coded SLA charts, side-by-side compare tool
Practical feasibility	Free, open tools; works as an add-on to existing complaint systems; runs on a laptop
Responsible design	Human-in-the-loop, minimal data, no auto-penalties, honest accuracy reporting
Quality of demonstration	Live end-to-end demo with genuine and fraud cases, plus a backup video

PS success metrics we will report

- Accuracy of before/after closure verification (measured on our labelled test set)
- SLA-adherence measurement accuracy against category deadlines
- Usefulness of hotspot detection (which wards/locations get flagged)
- Ease of complaint submission and reopen (number of taps / time taken)

16. How we build it in 24 hours

We build **step by step** with the Google Antigravity AI agent, testing after each step and saving with Git.

Step	Work	Done when
1	Tools set up (Node.js, Python, Git, Antigravity, Gemini API key)	All tools installed
2	Project brief + implementation plan approved	PROJECT_BRIEF.md in project folder
3	Backend: database, SLA engine, verification engine, API	http://localhost:8000/docs works
4	Test images + synthetic complaint data	Before/after/fraud images ready; ~200 complaints loaded
5	Frontend: report, track, task queue, review pages	Full flow works in the browser
6	Dashboard: map, heatmap, charts	Dashboard shows real numbers
7	Testing, bug fixes, UI polish	Demo runs start to finish without errors
8	PPT, demo script, backup video	Rehearsed at least twice

Suggested team split (4 members)

Member	Responsibility
A — Frontend	Complaint form, track page, task queue, UI polish
B — Backend	API, database, SLA engine, connecting everything
C — AI / verification	Gemini integration, EXIF/GPS/hash checks, accuracy testing
D — Data, dashboard & PPT	Test images, synthetic data, dashboard, presentation and demo video

17. Limitations & future scope

Honest limitations of the prototype

- Vision AI can be wrong in bad lighting or unusual angles — which is why humans make the final call
- EXIF metadata can be stripped or faked by a determined fraudster; it is one signal among five, not proof
- Messaging apps strip EXIF from genuine photos too, so the capture should happen inside the app
- Tested only on synthetic, team-created data, not on real municipal complaints
- No login or roles security in the prototype

What a real deployment would need

- In-app camera capture only (no gallery uploads) to block old or downloaded photos
- Dedicated AI-generated-image detection model and device attestation against GPS spoofing
- A trained garbage-detection model (e.g. YOLO) on real Indian civic images for better accuracy
- Integration API for existing systems (Swachhata / municipal portals) with official permission
- Login, roles, audit logs, and hosting on government cloud; regional language support
- Clustering (e.g. DBSCAN) for smarter hotspot detection; notifications to citizens and supervisors

18. Likely judge questions

Question	Our answer
What if the AI is wrong?	Every closure gets a confidence-based score and reasons, and flagged cases go to a human reviewer. The AI never takes action by itself.
Can't someone fake GPS or the photo?	We cross-check five independent signals (location, time, duplicate hash, metadata, visual content). Faking all of them together is much harder than faking one. In production, in-app camera capture closes the remaining gap.
Why not just trust the worker's photo?	Because of cases like Gurugram, where AI-faked photos were used to close complaints. A photo is only evidence if it is checked.
What is your accuracy?	We report the real number measured on our labelled test set (X out of Y correct) and are honest about its limits.
How would this scale?	Verification is a separate backend service that any existing complaint system can call through an API. SQLite would be replaced by a production database.
Why Gemini and not your own model?	In 24 hours, a vision LLM gives strong accuracy without training. A custom model is part of the future scope.
Does it respect privacy?	Yes — no personal details are stored, only the complaint, photo and location, and all demo data is synthetic.

19. Suggested PPT outline (slide by slide)

For the teammate building the PPT: about 12 slides, with little text and big visuals. Show more, write less.

Slide	Title	Content (from this document)
1	CleanProof	Name, tagline, PS-D04, team name and members
2	The problem	Tagline quote, SLA table, 'resolved but not resolved' photo (Section 1)
3	Real-world proof	Gurugram AI-faked photo case, no public SLA dashboard (Section 1)
4	Our solution	One-minute summary + comparison table (Section 2)
5	How it works	6-step flow diagram with the reopen loop (Section 4)
6	Verification engine	5 checks table + score bar (Section 6) — the most important slide
7	Architecture	Architecture diagram (Section 7)
8	Tech stack	Logos of React, Tailwind, Leaflet, FastAPI, SQLite, Gemini (Section 8)
9	Product screens	Screenshots: report form, review compare tool, dashboard
10	Responsible design	Human-in-the-loop, privacy, no auto-penalties (Section 13)
11	Impact & future scope	Who benefits + production roadmap (Sections 3 and 17)
12	Thank you / Live demo	Switch to the live demo (Section 14)

Tip: take screenshots of the real app for slide 9 once the frontend is built. Real screenshots make the PPT look professional and prove the prototype works.

20. References

- PS-D04 "Resolved, Allegedly" problem statement (Kalachakra 2K26, Digital Public Services Track)
- Swachhata-MoHUA official app — apps.apple.com/in/app/swachhata-mohua/id1124033628
- SBM-Urban 2.0 SLA guideline summary — righttoinformation.wiki/garbage-collection-street-cleaning-municipal-complaint-india
- MCG Gurugram terminates staff for AI-faked before/after photos and GPS spoofing, The420.in (Jul 2026) — the420.in/mcg-gurugram-terminates-four-employees-ai-image-fraud-gps-spoofing/
- User-reported closure/verification issues, Google Play — play.google.com/store/apps/details?id=com.ichangemycity.swachhbharat
- OpenStreetMap — openstreetmap.org | Google Gemini API — ai.google.dev